National University of Computer & Emerging Sciences, Islamabad
Department of Software Engineering

Assignment # 02

Structural Testing and Coverage Analysis of PX4 Autopilot

Course Code	SE3002	**Total Marks**	100
Team Size	Group of 3 students	**Submission**	Report + workbook + test code + coverage evidence
System Under Test	PX4-Autopilot v1.17.0		

Purpose of the Assignment

Core purpose: This assignment evaluates your ability to analyze an unfamiliar, production-scale software system and derive defensible white-box tests from its implementation. You are not given a pre-selected class or function list. Your team must determine what production logic is meaningful and testable, select an appropriate PX4 test level, justify the resulting test design, and provide reproducible structural-coverage evidence. A high coverage percentage, raw tool output, or AI-generated test code is not sufficient without your own analysis and defence.

Coverage goal: Achieve the maximum practically achievable structural coverage of meaningful PX4 business/control logic through student-designed tests. The target is 100% statement coverage and 100% decision/branch coverage for the business/control logic included in the team's analyzed scope. Where full coverage is not reasonably achievable, every remaining gap must be identified, investigated, and technically justified with evidence.

System Under Test and Fixed Baseline

System: PX4-Autopilot, an open-source autopilot stack for drones and autonomous vehicles. For this assignment, all groups must use the same stable baseline: **v1.17.0**. Do not use the moving main branch as your evaluated baseline.

Repository: https://github.com/PX4/PX4-Autopilot

```
git clone --branch v1.17.0 --recursive https://github.com/PX4/PX4-Autopilot.git
cd PX4-Autopilot
git rev-parse HEAD
```

- Record the exact commit hash produced by your local clone in the report. All coverage and test evidence must be traceable to this baseline plus your own test-related changes.
- Do not change production behavior merely to make the code easier to cover. Any production-code modification must be separately justified and clearly identified; testability changes must not alter intended behavior.
- No physical flight controller, drone, sensor, or real flight is required or expected for this assignment. Use local build/test facilities and simulation where needed.

Local Development and Execution Requirement

Local execution is compulsory. Every team must be able to clone, build, execute tests, and reproduce its coverage evidence on a student-owned system. The report must state the OS, hardware architecture, compiler/toolchain, and relevant PX4 build/test commands used.

- Ubuntu: use a currently supported PX4 Ubuntu development environment (22.04/24.04).
- Windows: use the supported WSL2 development route; do not rely on Windows-only GUI tooling.
- macOS: supported on both Intel and Apple Silicon Macs using the PX4 macOS development environment.
- QGroundControl may be used as an optional visualization or interaction aid, but the system under test and all claimed structural coverage must be PX4 production code.

Tools and Technology

- Development/build environment: Use the official PX4 toolchain appropriate to your operating system. You may use a suitable IDE/editor, but command-line build/test evidence must remain reproducible.
- GTest unit tests: Appropriate for code with minimal, internal-only dependencies.
- GTest functional tests: Appropriate when the code under test depends on PX4 parameters, uORB messages, or more advanced test support.
- SITL unit tests: Appropriate only where the selected logic requires full PX4 flight-controller context such as time, drivers, or complete SITL components.
- Coverage: PX4 provides a tests_coverage target using gcov/lcov. Equivalent coverage tooling such as llvm-cov is acceptable when needed by the local platform. Required structural coverage applies to the selected production business/control logic only; pure GUI/front-end presentation files are outside the assessed coverage scope.
- MC/DC: MC/DC is not required across the complete repository or for every compound decision. Each group must select one safety-, mission-, or control-critical class or cohesive business/control-logic component and justify why the selected component is critical. Define the critical behaviour being analyzed, then apply MC/DC to the non-trivial compound decisions that implement or directly govern that critical behaviour. For each applicable decision, explicitly identify the atomic Boolean conditions and the test pairs that demonstrate each condition's independent effect on the overall decision outcome. Unrelated compound decisions elsewhere in the repository, and trivial logging, wrapper, generated, test-only, or presentation logic, are not MC/DC targets. Tool-supported MC/DC may be included, but it does not replace the required derivation.
- Condition coverage / condition-decision coverage: These are not separate whole-scope reporting targets in this assignment. For every compound decision analyzed for MC/DC within the selected critical component, the test set must show each relevant atomic condition taking both True and False values and the overall decision taking both True and False outcomes. Record this evidence in the MC/DC matrix in the testing workbook rather than as an additional coverage percentage.
- Optional techniques: Fuzz testing or simulation/integration tests may supplement the submission where appropriate, but they do not replace the required structural test design and MC/DC analysis.

Useful official resources: PX4 Repository | Developer Environment | Unit Tests | Testing & CI | Simulation | v1.17 Release Notes

Existing PX4 Tests and AI-Assisted Work

Existing upstream tests may be read and run to understand PX4 conventions and establish a baseline, but they do not count as student-authored tests. Copying, lightly renaming, or mechanically reproducing upstream tests is not an acceptable submission. Your team must identify its own test changes and explain how the tests were derived from production logic.

- Submit a clear Git diff/patch of your own changes against v1.17.0, in addition to the created/modified test files.
- AI-assisted coding or code-navigation tools may be used. The team remains responsible for every selected target, condition, expected result, test, coverage claim, and conclusion.
- Include a brief AI-assistance record identifying material uses of AI (for example: repository navigation, build troubleshooting, test scaffolding, code suggestions) and any important assumptions it introduced. Do not submit raw chat transcripts unless specifically requested.
- “The tool generated it” or “the AI said so” is not a defensible test basis. During the viva, any member may be asked to explain or modify any submitted test.

Required Workflow

1. **Establish the fixed baseline.** Clone PX4-Autopilot v1.17.0 recursively, record the commit hash, and set up a supported local development environment.
2. **Build and execute the baseline.** Demonstrate that PX4 and its existing test infrastructure can run locally before adding your own tests. Record the commands and evidence.
3. **Analyze the repository.** Explore production code and identify meaningful business/control logic for structural testing. Pure GUI/front-end presentation files, visual rendering, styling, and interface-only code are outside the required scope. Do not restrict yourself to trivial getters, setters, wrappers, generated code, or test-only utilities.
4. Build the structural test basis. For the business/control logic you claim as your analyzed scope, identify executable statements, decision outcomes, dependencies, state, parameters/messages, and any setup needed to control them. For

MC/DC, select and justify one critical class or cohesive business/control-logic component, define the critical behaviour being analyzed, and identify the non-trivial compound decisions that directly implement or govern that behaviour.

5. Design the test suite. Derive tests for statement coverage and decision/branch coverage across the selected business/control logic. In addition, derive MC/DC tests for the applicable critical compound decisions within the selected MC/DC component. Include boundary, invalid, error, and state-transition cases where the source logic makes them relevant.
6. **Implement and execute the tests.** Use GTest unit, GTest functional, or SITL testing as justified by the dependencies of the code under test.
7. Measure and iterate. Collect a baseline coverage view, execute your tests, measure the resulting coverage, and continue adding or refining tests to achieve the maximum practically achievable coverage of the selected business/control logic. Do not stop merely because a minimum percentage has been reached. Continue until the stated targets are met or only specific, investigated, and defensible gaps remain.
8. **Analyze and defend.** Explain coverage gaps, investigate failures/blockers, identify confirmed defects if any, and reach a scoped conclusion about the adequacy of your structural test suite.

Assignment Tasks

Part 1 — Repository Analysis and Structural Test Basis (20 marks)

Begin from the production code, not from a desired coverage percentage. Your report must show that you understood the selected behavior and its dependencies before writing the tests. Keep the report evidence-focused: do not duplicate information that is already clearly demonstrated by test source code, test-framework output, or the generated coverage report.

- Provide local setup/build evidence and identify the fixed tag, commit hash, OS, compiler/toolchain, and relevant build/test commands.
- Identify the production areas you chose to analyze and explain why they are non-trivial and suitable for white-box testing.
- Explain important dependencies such as parameters, uORB messages, state, time, vehicle mode, or simulator context, and justify the test level chosen for each target.
- Identify decision points and compound decisions directly from the business/control implementation. Pure GUI/front-end presentation code, generated code, test code, pure accessors, and trivial wrappers are outside the assessed structural-coverage scope and must not be used to make the selected scope appear substantial. You are not required to transcribe every decision into a separate inventory table; the analysis should be visible through the selected scope, compact test inventory, MC/DC evidence, and executable tests.

Scope selection record

In the report, identify the selected production areas in concise prose or bullets. For each area, state the production file/class/component, its responsibility or critical behaviour, key dependencies/state, why it is non-trivial and included, and the PX4 test level you plan to use. A separate scope or decision-inventory table is not required.

Part 2 — Structural Test Derivation and MC/DC Analysis (30 marks)

Derive the test suite from the business/control implementation identified in Part 1. The number of tests is not prescribed; the structural obligations determine how many tests are required. Statement and decision/branch coverage apply across the selected business/control scope. MC/DC is focused on one justified critical class or cohesive business/control-logic component and the non-trivial compound decisions that implement or directly govern its selected critical behaviour.

- Statement coverage: show which tests execute the relevant executable statements in the selected business/control logic.
- Decision/branch coverage: exercise both outcomes of each reachable decision in the selected business/control logic.
- MC/DC: select one safety-, mission-, or control-critical class or cohesive business/control-logic component. Justify why the component and the selected behaviour are critical. Within that component, identify the non-trivial compound decisions that implement or directly govern the critical behaviour and perform MC/DC for those decisions. For each analyzed decision, identify its atomic Boolean conditions and demonstrate that each condition can independently affect the overall decision result. The selected component must contain sufficient non-trivial decision logic to make the analysis meaningful; a trivial wrapper or a component with only a superficial compound check is not acceptable.

- Condition evidence within MC/DC: For each compound decision analyzed within the selected critical component, your test set must show every relevant atomic condition evaluating to both True and False and the complete decision evaluating to both True and False. Do not submit a separate condition-coverage or condition/decision-coverage percentage.
- Where short-circuit evaluation, state, parameters, time, messages, or mode transitions affect condition reachability, make the required setup explicit in the test code and summarize it in the compact inventory when it materially affects understanding.
- Include edge, boundary, invalid/error, and exceptional-state tests wherever the source code contains behavior that depends on such values or states.

Use ordinary execution statuses (for example PASS, FAIL, or BLOCKED) only where they accurately describe what occurred. There is no required minimum number of failed or blocked tests, and students must not manufacture failures.

Required Testing Workbook (.xlsx)

Keep structured test evidence in one spreadsheet workbook rather than reproducing test-case forms in the report. Teams may work in Microsoft Excel or Google Sheets; the final submission must be exported as .xlsx. The workbook must contain no more than two sheets.

- Sheet 1 — Test Inventory: one row per student-authored test. Keep it compact: Test ID, component/function, purpose or scenario, key controlled input/state, expected result, execution result, structural-coverage target, and test-file/evidence reference.
- Sheet 2 — MC/DC Evidence: provide the Boolean matrix for the selected critical component. For each analyzed compound decision, record the atomic-condition values, overall decision outcome, independence pair, condition demonstrated independent, and source/test reference.
- Do not create separate scope, coverage-gap, defect, or traceability sheets. Explain scope selection, remaining coverage gaps, limitations, and any genuine defects found in concise prose in the report, with references to the relevant test/code/coverage evidence.

The executable test code is the authoritative detailed record of fixtures, setup, execution logic, mocks, assertions, and teardown. Do not duplicate those details as long step-by-step procedures in either the report or workbook.

Part 3 — Test Implementation, Execution and Coverage Evidence (35 marks)

A. Test implementation and execution (15 marks)

- Implement your own test code using the PX4 mechanism justified for each target. The test suite must compile and execute from a documented local setup.
- Use clear, deterministic setup–run–check structure. Control relevant state and dependencies rather than relying on accidental ordering or previously executed tests.
- Any required CMake/test registration changes must be included in your submitted patch. Avoid production-code changes; if a testability change is unavoidable, document it separately and show that intended behavior is unchanged.
- Existing PX4 tests may be used to establish the baseline or understand patterns, but your claimed test contribution must be identifiable as student-authored code.

B. Structural coverage measurement and analysis (20 marks)

- Capture coverage before adding your own tests where practical, then capture final coverage after your tests. Explain what your suite contributed for the selected business/control logic. Pure GUI/front-end presentation files are not part of the required coverage scope.
- Coverage goal: achieve the maximum practically achievable structural coverage of the meaningful business/control logic in your analyzed test scope. The target is 100% statement coverage and 100% decision/branch coverage for that scope. Coverage percentage is not a stopping criterion: teams are expected to investigate uncovered code and continue designing or refining tests while additional defensible coverage can be achieved. Complete MC/DC evidence is required for the applicable critical compound decisions within the one critical class or cohesive component selected and justified in Part 2.
- The entire PX4 repository is not expected to reach 100% coverage. The targets apply to the substantial business/control production-code scope that your group selected and analyzed from the whole repository. Pure GUI/front-end presentation code is excluded from the required scope; exclusions must not be used to omit difficult business/control logic merely because it is hard to test.

- Coverage must be reproducible from the submitted test code and commands. Screenshots alone are insufficient; include the generated report or machine-readable/HTML evidence.
- If full coverage is not achievable, every remaining gap must be identified and investigated. Identify the exact uncovered statement/branch/condition, show why it is not reachable or practical in the local test environment, and state what additional environment, dependency, or test strategy would be required. “Hardware dependent” by itself is not a sufficient justification, and difficult-to-test logic must not be excluded merely to improve the reported percentage.

Suggested PX4 commands (adapt as needed for your platform):

```
make tests
make tests TESTFILTER=<regex>
make tests_coverage    # gcov/lcov coverage target where supported
make px4_sitl          # build PX4 SITL
```

Coverage reporting: Submit the generated baseline and final coverage reports and state the key statement/line and branch results for each analyzed scope item in concise prose or annotated report excerpts. Do not create a separate summary table merely to copy percentages already visible in the coverage tool output.

Part 4 — Findings, Coverage Gaps and Final Quality Judgment (15 marks)

Interpret the evidence rather than ending the report with a percentage. Your conclusions must remain limited to what your team actually analyzed and executed.

- Investigate every FAILED or BLOCKED test before claiming a software defect. Check the test setup, expected result, state, data, environment, and dependency behavior first.
- If structural testing reveals a confirmed, reproducible implementation defect, discuss it briefly in the report and reference the relevant test ID and evidence. Include the affected source location, reproduction conditions, expected result, and actual result. No defect-management tool and no separate defect table are required; there is no minimum defect count.
- Explain the most important coverage gaps, unreachable or defensive code, environmental limitations, and the risks that remain outside your tested scope.
- Suggest concrete improvements: additional test levels, alternative state setup, simulator support, dependency control, instrumentation, or refactoring for testability where justified.

Final quality judgment (300–400 words): State what the structural evidence supports about the tested PX4 production logic, what remains unsupported, how much confidence the achieved coverage provides, and which limitations prevent a broader conclusion. Do not claim that PX4 as a whole is “high quality” or “fully tested” from the evidence of this assignment.

Submission Checklist

- One concise report containing group details, local environment, fixed tag/commit, repository analysis and scope justification, testing approach, MC/DC component selection and interpretation, coverage analysis, confirmed defects if any, remaining gaps/limitations, and the 300–400 word final judgment.
- Student-authored/modified test source files and any required CMake/test-registration changes.
- A Git diff/patch against PX4-Autopilot v1.17.0 so that the submission can be reconstructed without uploading the unchanged repository.
- Setup/run instructions with exact commands needed to reproduce the tests and coverage on the team’s environment.
- Baseline and final coverage evidence. Submit the generated coverage report (HTML or equivalent) and include concise screenshots/excerpts in the report where useful.
- Test execution evidence showing the submitted tests actually ran. Include relevant logs/output, not only screenshots of source code.
- One testing workbook (.xlsx) with no more than two sheets: Test Inventory and MC/DC Evidence. Teams may collaborate in Google Sheets or Excel, but must submit the exported .xlsx file.
- Brief AI-assistance record identifying material AI use and any assumptions introduced through it.
- All submitted files must follow the naming convention <Rollnumber1_Rollnumber2_Rollnumber3_Section>.<extension> unless the LMS specifies a different convention.

Demo/Viva Requirement

Important: submission of files does not guarantee full marks. During the demo/viva, any group member may be asked to build or run a submitted test, locate the corresponding production decision, identify atomic conditions, explain an MC/DC independence pair, justify the selected PX4 test level, interpret a coverage gap, or predict what coverage would be lost if a test were removed. Each member must understand the complete submission.

Evaluation Rubric

Component	What is being assessed	Marks	CLO mapping
Repository analysis and structural test basis	Correct fixed baseline and local setup; meaningful non-trivial production-code selection; accurate understanding of responsibilities, dependencies, decisions, conditions, state, and appropriate PX4 test level.	20	CLO3
Structural test derivation and MC/DC	Defensible statement/branch obligations across selected business/control logic; justified selection of one critical class or cohesive component; correct identification of the critical behaviour and its applicable compound decisions; valid atomic-condition analysis and MC/DC independence pairs; appropriate edge/boundary/error/state cases; complete traceability from production logic to tests.	30	CLO2
Test implementation and execution	Student-authored tests compile and run; correct use of GTest unit, functional, or SITL mechanisms; deterministic setup–run–check structure; reproducible execution evidence and clean test integration.	15	CLO2
Coverage measurement and gap analysis	Authentic baseline/final coverage evidence; maximum practically achievable structural coverage pursued; 100% statement and decision/branch coverage reached where feasible for selected business/control logic; remaining gaps precisely investigated and technically justified; MC/DC evidence correctly demonstrates the applicable critical compound decisions within the selected critical component.	20	CLO3
Findings and final quality judgment	Failures/blockers investigated correctly; any confirmed defects documented concisely and supported by reproducible evidence; limitations and residual risk recognized; final conclusion is scoped, evidence-based, and does not overclaim.	15	CLO3

The demo/viva deduction rule applies across all rubric components. Marks may be reduced when submitted analysis, code, tests, coverage evidence, or conclusions cannot be explained or reproduced by the students.